

</PROMPT ENGINEERING FRAMEWORK

Prompt-engineering i praksis

Fra hverdagslig kognitiv forsterkning til deterministisk systemintegrasjon og
prototypegenerering

Innholdsfortegnelse

Verdiskaping for mennesker

Hvorfor KI er viktig for industrien

Hva bruker vi KI til & vår dialog med KI

Prompts vs Instruktivt prompts

Instuktive prompts

Planleggingsfasen av LLM modeller

Strategisk modellvalg

Prototyping med KI - Karriere-KI

Multi-Model Peer Review

Automatiske prompts

Hvilken verdi har KI for oss

Kognitiv forsterkning

Å løfte enkeltpersoner fra utførelse til strategi

- **Avlasting av rutineoppgaver** - Automatisk oppsummering, tekstutforming og kontekstsyntese.
- **Rask kontekstforståelse** - Umiddelbart uttrekk av nøkkelinnsikt fra omfattende tekniske manualer og logger.
- **Fokus og flyt** - Reduserer kognitiv friksjon og beslutningstretthet gjennom arbeidsdagen.

Interaktiv partner

Alltid tilgjengelig sparringspartner

- **Inspirere til kreativitet** - assisterer konstruktive tilbakemeldinger.
- **Se flere perspektiver** - Test antakelser og utforsk uventede scenarier før du setter i gang.
- **Effektiv læring** - Rask tilegnelse av kunnskap innen nye fagfelt og verktøy.

Hvorfor KI er viktig for industrien



Akselert fremdrift

Reduserer tiden fra prosjektplanlegging til ferdig produkt fra uker til timer ved å automatisere oppsett, maler og rutinearbeid.



Standardisert Norsk kvalitet

Sikrer at alle i bedriften kan levere med samme presisjon og standard, uavhengig av spesialkompetanse.



Skalerbar ingeniørkunst

Løfter små team til å bygge trygge systemer i storskala, ved å automatisere tunge og avanserte utviklingsløp.

Hva bruker vi KI til



Arbeidsflytkategorisering

Rutineoppgaver -

Enhetstesting, kodeløft, endringssammendrag og formatkonvertering ved hjelp av KI-modeller

Komplekse oppgaver -

Systemdesign, generering av sammensatte kodestrukturer, automatiseringer, sikkerhetsrevisjon og arkitekturprototyping ved hjelp av tunge resonneringsmodeller.

Prompts vs konstruktive prompts

Lær av erfaringer

- Når vi ikke tar oss tiden til å tenke gjennom prompts, blir promptene våre seende slik ut.
- Dette tar dobbelt så lang tid, som vi tror, som en konsekvens av at KI må gjette og øker risiko for hallusinasjoner.

> Eksempel på ikke konstruktivt bruk av KI.

```
# Dagligdags tale
"vør så snill å utvikle en programvare som forklarer
matematiske formler.."
"Kan du gi meg tilbakemelding på koden"
"Koden min funker ikke, hva er problemet.."
```

Lær av erfaringer

-
- **Prosesseringstid** - tiden det tar fra KI får instruksjonene til programmet er laget er lav.
- **Ressursvennlig** - Vi fjerner all støyen fra grammatikken.
- **KI-prompting** - KI-en klarer raskt å skille mellom nøkkelord fra verdien på instruksjonene

> Instruktiv @ syntaks stil ved bruk av KI

```
# Instruktivt språk
@ROLE: AI Engineer
@GOAL: "Refactor Karriere-KI- draft file & enforce strict
type definitions"
@TARGET: "src/controllers/auth.ts"
@REQUIRE: ["Validate input against JSON schema" , "Apply
clean code principles & async patterns"]
@FORMAT: JSON
```

Konstruktive prompts

Strukturert & Dokumentert

- **Dokumentasjon** - Lettleselig for både mennesker og KI
- **Prosesseringstid** - Rask og krever mindre gjetting
- **Ressursvennlig** - Balanse mellom detaljer og stikkord
- **KI-prompting** - Nå kommuniserer vi på data som LLM-er er trent på.
- **Områder**- prototyping, dokumentasjon, prosjektbeskrivelser..

>_ Eksempel prompt

```
# Standard strukturert Markdown
# Rolle / Ekspertise
Lead AI Engineer

# Oppgave
Refaktorer path/to/Karriere-KI sin kladd-fil og sikre
streng typesikkerhet (strict type definitions) i ...
```

Enterprise kontrakter

- **Dokumentasjon** - Tekniske systemspesifikasjoner og API-kontrakter.
- **Prosesseringstid** - Høy
- **Ressursvennlig** - Krever tokens for skjemabeskrivelsen
- **KI-prompting** - separering av prompter, tvinger vi KI-en til å svare nøyaktig slik programvaren eller pipinen krever
- **Områder**- Miljøer hvor prompten skal være stabil, som automatiserte skript, produksjonskode..

>_ Eksempel prompt

```
# Enterprise-contracts
[Systemprompt]
Du er en AI Engineer. Svaret SKAL utelukkende leveres som
et gyldig JSON-objekt basert på det forespurte skjemaet

[Brukerprompt]
...
```

Planleggingsfasen av LLM-modeller



Brukerbehov

Kartlegging av behov

- Automatisere eller LLM?
- Streng logikk?
- Kreativitet?
- Effektivitet?
- Håndtere store mengder data?



Måltrettet sammenligning

sammenligner fordelene til de aktuelle modellene, begrensninger, kostnader og benchmark-resultater mot behovene



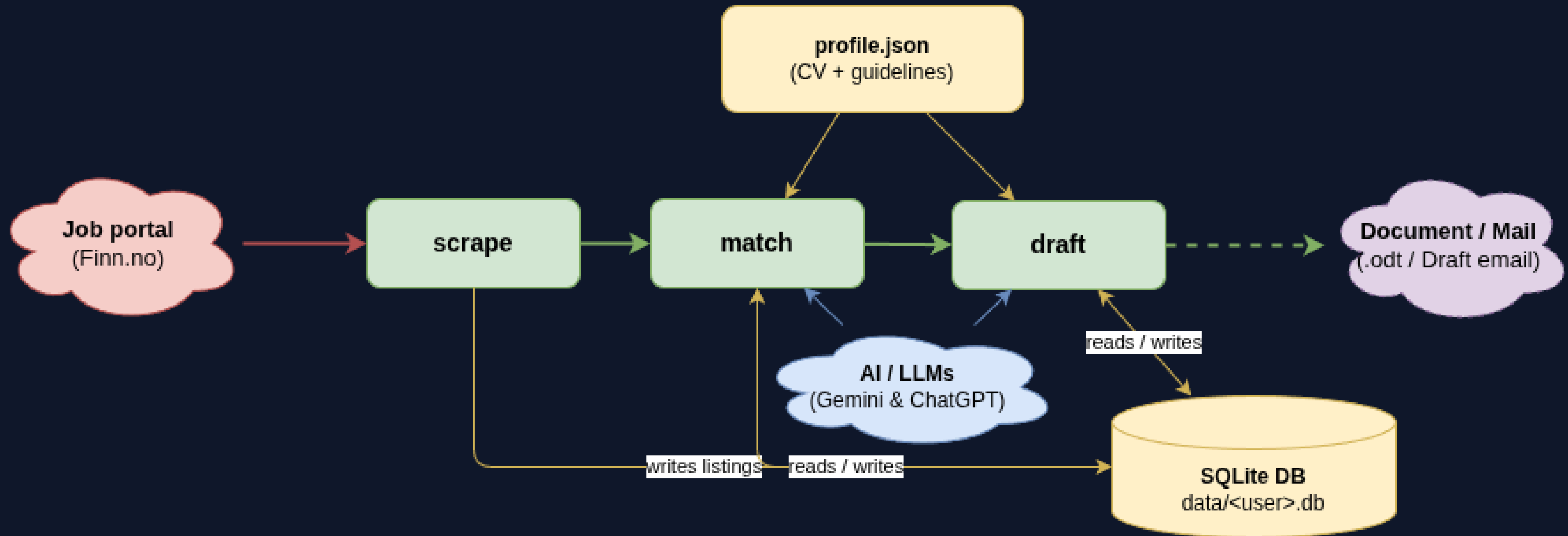
Modellvalg basert på prosjektets rammer

Velger den mest optimale og kostnadseffektive modellen som tilfredsstiller kravene..

Oppgavekategori	Kriterier for valg	Foretrukket modell	Hovedfordel
Rutineoppgaver Formatering, Repitivkode	Redusert responstid? Stor dokumentasjons kontekst? Lave tokenkostnader? Språk?	Gemini Flash GPT-4o-mini	LLM-modellene er kjent for sin korte responstiden, og er optimalisert for automatisering i storskala
Kompleksarkitektur som system design, refaktorering	Avansert logikk som matte eller dyp resonnering, instruksjonsmargin?	Claude Sonnet	LLM-modellen er kjent for sin overlegen logikken sin, kodegenerering og strukturert presisjon
Dypt kontekstsøk dokumentanalyser, kodebase analyser	Gigantisk kontekst?	Gemini Pro	Er kjent for å kunne prosessere gigantiske prosjektmapper og datasett i en enkelt prompt og har kontekstkapasiteten sin på +1 mill tokens og er fortsatt presis i filtreringen sin.

Prototyping med KI - Karriere-KI

Karriere-KI — Pipeline



Prototype Prompting

Prompteksempel for å generere prototypen

Konstruktiv bruk av Claude Code (DEMO)

Prompts for developer tools

Tech Stack & Architecture

Build a Python 3.14 CLI app that scrapes finn.no, scores jobs using GPT-4o with Cubic scaling as progressive algorithm for scoring, and drafts cover letters using Gemini.

Requirements

- DB: SQLite (async, jobs table: id, title, company, description, score, status, draft)
- Auth: User profiles loaded dynamically from resources/profiler/<username>.json
- CLI commands: scrape, match, list, detail, draft, status

Karriere-KI

Hva er det?

Karriere-KI er en prototype utviklet i samarbeid for å forenkle jobbsøking:

- **Innsamling** - Terminal-app som skrapper portaler som **arbeidsplassen** og **finn** og lagrer stillinger i en database.
- **Matching** - LLM sammenligner en strukturert JSON-profil med stillingen for å vurdere egnethet. **Utkast** - LLM kobler stillingens harde og myke krav opp mot en brukerprofil.
- **Automatisk pipeline** - Utviklet en Automatisk pipeline som starter fredager kl 12:00:00
- **Automatisk logging** - Pipelinen logger alle events.

Karriere-ki



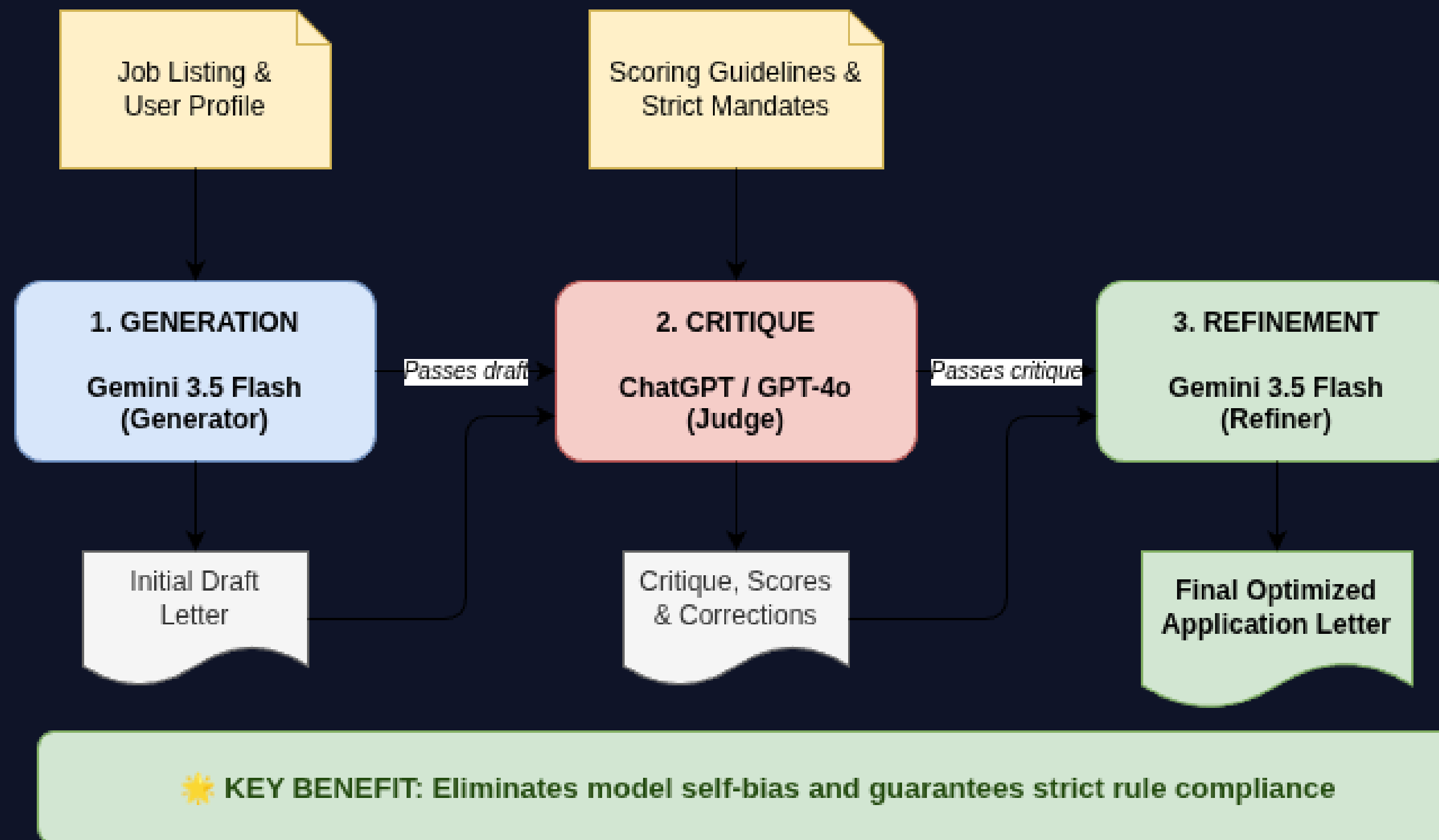
Karriere-KI

Hensikten med karriere-ki

- fremheve verdien av kandidaten for bedriften
- å effektivisere søknadsvurderingen for rekrutterere
- Å sikre mer konkrete, relevante og dokumenterbare søknader

Multi-Model Peer Review

Multi-Model Peer Review (LLM-as-a-Judge) Architecture Pattern



Fordeling av ansvaret for verktøy & LLM-motor

Interaktiv partner

Utviklerverktøy

Vi valgte Claude Code som en konsekvens av sin overlegne kode og kontekstforståelsen, samt den minimale hallusinasjonsraten for MVP-modeller.

Konstruktiv bruk av Claude Code (DEMO)

Prompt demonstration

Tech Stack & Architecture

Build a Python 3.14 CLI app that scrapes finn.no, scores jobs using GPT-4o with Cubic scaling as progressive algorithm for scoring, and drafts cover letters using Gemini.

Requirements

- DB: SQLite (async, jobs table: id, title, company, description, score, status, draft)
- Auth: User profiles loaded dynamically from resources/profiler/<username>.json
- CLI commands: scrape, match, list, detail, draft, status

LLM-motorer

GPT & GEMINI

I stedet for å lene seg på tradisjonell automatisering, utnytter systemet intelligensen til flere LLM-er (**Gemini modeller** og **GPT modeller**) til å autonomt **poengsette stillingstreff** i forhold til **personlige data**, **utforme tilpasset søknader** og gi konstruktive tilbakemeldinger.

LLM models - Brukt av applikasjonen

Gemini

Hvorfor?

Valgt for å samle inn data takket være modellens store kontekstvindu, til å analysere store mengder dokumentasjon i en operasjon.

Ansvarsområder

- Samle inn data, fra prosjekter, kompetanse profil
- Fokusere på verdien av prosjektet
- Integrere kritikken fra OpenAI for å finpusse utkastet.
- Generere søknadsutkast med fokus på verdi.
- Analysere Stillingens krav opp mot kompetanse profil

GPT

Hvorfor?

Valgt som uavhengig korrekturleser (peer review) for å unngå "egensensur" under evalueringen.

Ansvarsområder

- Evaluere og poengfordele kandidatens match mot stillingen basert på brukerdata.
- Identifisere forbedringspotensial i søknadsutkastet.
- Evalvere søknader gjennom 3 roller CTO, Senior Utvikler, Rekruter

Progressive algoritmen

GPT-40-mini blir også brukt for å håndtere algoritmen **kubisk vekting**: $5 \times (H/5)^3$ for å få et objektivt syn på evaulering av ferdigheter (H) opp mot kravene i annonsen.

> Eksempel

```
3 av 5 treff gir kun 1,1 poeng (22% score), mens 5 av 5 gir full pott på 5,0 poeng. Dette straffer ufullstendig match og løfter frem arbeidsgivere som kandidaten har best match mot.
```

Automatiske prompts

Hvordan karriere-ki sender inn prompter

>_ Forenklet struktur for generering av ferdigstilte utkast

```
# Enterprise prompts for Gemini-flash
[Poler]
Draft:{first_draft}
Critique:{critique}
Profile:{guidelines_text}
[Format]
Score:[N/10]
Subject:[Text]
[Body]
[Regel]
100% fakta.Start:"{mandatory_opening_paragraph}"
```

Hvordan karriere-ki sender inn prompter

>_ Eksempel på GPT-4o-mini struktur

```
# Enterprise contract prompts for gpt-4o
[Kritikk]
Draft:{first_draft}
Job:{job_description}

[Format]
Critique:[Points]
Score:[N/10]

[Regel]
Sjekk krav, stil, og forbudte ord (lidenskapelig).
```


Konklusjon

For å unngå hallusinasjoner og lang prosesseringstid, må vi bytte ut den dagligedagse talen til en konstruktiv prompt-metodikk

Valget av prompt-metodikk handler om å balansere skrivehastighet i chatten, lesbarhet for dokumentasjon og streng formatlåsing for automasjon.

For å unngå begrensningene i synkron tenking, kan ikke vi bare bruke en modell til alt. Vi må forstå de ulike verdiene for de ulike modellene (f.eks. raske Gemini-flash for rutiner, og Claude Sonnet for tung logikk).

Når vi strukturerer prompter som konstruktive prompter og lar LLM-er evaluere hverandre, forvandles KI fra et eksperiment til skalerbar ingeniørkunst av standardisert kvalitet.

Karriere-ki deler opp prosessen, fremfor å be KI gjøre alt i en operasjon. Bruk av en matematisk formel sikrer at KI-en tvinges til å levere ærlige resultater, og forhindrer at dårlige matcher skjules bak de sosiale filtere.

Tusen takk !

Utforsk et minutt demonstrasjonsvideo på hvor effektivt en konstruktiv KI-prompting kan være & kildekode under.



Videodemonstrasjon

Trykk på ikonet for å se en demonstrasjon av konstruktiv Prompting.



Kildekode

Trykk på ikonet for å se kildekoden til demonstrasjonen.